

CSCE 611 OPERATING SYSTEMS · MODULE 03 · LECTURE NOTES

CPU Scheduling

Deciding who runs, with no idea what will happen next.

MODULE OUTCOMES

- State the scheduling objectives and show they conflict.
- Analyse FIFO, SJF, round robin, and MLFQ.
- Explain why MLFQ approximates SJF without knowing job lengths.
- Explain priority inversion and inheritance.
- Explain how real schedulers handle multicore and fairness.

1 Objectives, and why they conflict

Metric	Definition	Who cares
Turnaround time	Completion time minus arrival time.	Batch work: compiles, renders, analyses.
Response time	First time scheduled, minus arrival time.	Interactive use. A shell that takes 200 ms to echo a keystroke is unusable regardless of its throughput.
Throughput	Jobs completed per unit time.	Servers and batch systems.
Fairness	Each runnable job receives a proportional share.	Multi-user and multi-tenant systems.
Predictability	A bounded worst case.	Real-time systems, where a late answer is a wrong answer.

THEY PULL AGAINST EACH OTHER

Minimising **turnaround** means running short jobs first, which requires knowing which are short and which starves the long ones.

Minimising **response time** means switching often, so everything gets attention quickly.

Maximising **throughput** means switching rarely, because each switch costs a cold cache and a TLB flush (Module 02) — the direct opposite.

Fairness forbids systematically preferring one class of job, which rules out pure shortest-first.

No policy optimises all of them. Judging a scheduler means identifying which trade it has taken and whether that matches the workload.

2 The basic policies

Policy	Rule	Strength	Failure
FIFO	Run to completion in arrival order.	Trivial; no starvation.	Convoy effect: one long job at the front delays everyone. Average turnaround is terrible.
SJF	Run the shortest job first, to completion.	Provably optimal average turnaround.	Requires knowing job lengths. Starves long jobs. A long job that arrives first still blocks everything.
STCF	Preemptive SJF: switch when a shorter job arrives.	Handles arrivals during execution.	Still needs the future; still starves.
Round robin	Fixed quantum; rotate through runnable jobs.	Excellent response time; obviously fair.	Poor turnaround — everything finishes at roughly the same late time. Quantum too small and switch overhead dominates.

SJF DEFINES THE TARGET, NOT THE SOLUTION

Shortest-job-first minimises average turnaround time, and that is a theorem rather than an observation.

It is also unimplementable: the kernel cannot know how long a process will run. And it starves long jobs indefinitely under a steady arrival of short ones.

So the question this module actually answers is: *can a scheduler behave like SJF without knowing the future, and without starvation?* The answer is to infer job behaviour from its past, which is MLFQ.

3 Multi-level feedback queues

Maintain several queues at descending priority.

```

RULE 1  A job in a higher queue runs before any job in a lower one.
RULE 2  Jobs in the same queue run round robin.
RULE 3  A new job enters at the HIGHEST priority.
RULE 4  If a job uses its entire quantum, it drops one level.
RULE 5  If a job yields before the quantum expires, it stays.
RULE 6  Periodically, move ALL jobs back to the highest queue.
```

Rules 4 and 5 do the work. A job that blocks for I/O before its quantum expires is interactive and keeps its high priority, so it responds quickly. A job that burns a full quantum is CPU-bound and sinks, so it does not interfere with interactive work.

WHY THIS IS SJF WITHOUT CLAIRVOYANCE

Short and interactive jobs block early and often, so rule 5 keeps them near the top and they are scheduled first — which is what SJF would have done.

Long CPU-bound jobs consume whole quanta and sink under rule 4, so they run only when nothing shorter is waiting — again what SJF would have done.

The scheduler has reproduced SJF's behaviour by **observing the past instead of predicting the future**, which is a general and transferable idea.

3.1 Rule 6 is the one people forget

Without the periodic boost, a long-running job that has sunk to the bottom never rises again. Under a steady stream of short interactive jobs it starves permanently.

The boost also handles jobs that **change character**: a compiler that spends ten minutes CPU-bound and then begins waiting for input should be treated as interactive from that point. Rule 6 gives it the chance to demonstrate its new behaviour.

GAMING THE SCHEDULER IS A REAL ATTACK

A process that issues a trivial I/O call just before its quantum expires never *fully* consumes a quantum, so rule 4 never demotes it. It sits at top priority indefinitely and monopolises the CPU.

The fix is to account for CPU time **cumulatively** rather than per quantum: track the total time a job has used at its current level and demote when that budget is exhausted, however the usage was divided up.

The general lesson applies well beyond scheduling: any policy that infers intent from observed behaviour can be manipulated by producing the behaviour deliberately. Measure cumulative resource consumption, not individual events.

4 Priority inversion

```
LOW  acquires lock L and begins its critical section.
MED  becomes runnable and preempts LOW (higher priority).
HIGH becomes runnable, tries to take L, and BLOCKS.
```

```
HIGH is now waiting for LOW.
LOW cannot run because MED preempts it.
MED, which never touches L, is effectively blocking HIGH.
```

The priority ordering is inverted, and it persists for as long as MED has work to do.

MARS PATHFINDER, 1997

The Pathfinder lander began resetting itself repeatedly on the Martian surface. A high-priority bus management task blocked on a mutex held by a low-priority meteorological task, while a medium-priority communications task kept the low-priority one off the CPU. The watchdog timer observed the high-priority task missing its deadline and reset the system.

The VxWorks kernel supported priority inheritance; it had been disabled. Engineers reproduced the fault on the ground replica, identified the inversion, and uploaded a patch enabling inheritance — to Mars.

It is the standard example because the diagnosis and the fix are both exactly this module's material, and because it demonstrates that scheduling subtleties have consequences beyond benchmark numbers.

4.1 Inheritance and ceilings

Mechanism	How	Effect
Priority inheritance	When a high-priority task blocks on a lock, the holder is temporarily raised to the blocked task's priority until it releases.	The holder now preempts medium-priority tasks, finishes its critical section promptly, and the inversion window is bounded by the length of that critical section rather than by unrelated work.
Priority ceiling	Each lock carries the highest priority of any task that may acquire it. A task is raised to that ceiling as soon as it takes the lock.	Prevents inversion before it starts and also prevents certain deadlocks (Module 05). Requires knowing in advance which tasks use which locks.

5 Real schedulers

System	Policy	Idea
Linux CFS	Completely Fair Scheduler.	Track each task's <i>virtual runtime</i> — CPU time received, scaled by weight — and always run whichever has the least. There is no fixed quantum: a task runs until it is no longer the least advanced. Fairness becomes a definition rather than an approximation.
Linux EEVDF	Earliest Eligible Virtual Deadline First.	CFS's successor, adding explicit latency targets so that interactive tasks get bounded response without sacrificing fairness.
Windows	Priority-based with dynamic boosts.	Threads get priority boosts on I/O completion and for foreground windows, decaying over time — tuned heavily for perceived interactive responsiveness.
EDF	Earliest Deadline First.	Real-time. Provably schedules any feasible task set on one CPU, provided total utilisation does not exceed 1. Used where deadlines are hard.

5.1 Multicore

PER-CPU QUEUES, AND A NEW TRADE

A single global run queue requires every CPU to take the same lock on every scheduling decision — exactly the contended-atomic scalability failure of CSCE 614 Module 11. Real kernels use per-CPU run queues with periodic load balancing.

That creates a tension. **Cache affinity** argues for keeping a process on the CPU whose caches hold its working set. **Load balancing** argues for moving it to an idle CPU.

Migrating costs a cold cache, which Module 02 noted is often the dominant context-switch cost — frequently more than the idle time the migration was meant to recover. On a NUMA machine it is worse still, because the process's pages remain on the old node and every access becomes remote (CSCE 614 Module 07).

Real schedulers therefore migrate reluctantly, with hysteresis, and expose affinity controls for when the administrator knows better.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
OSTEP — Chapters 7–10 (Scheduling, MLFQ, Lottery, Multiprocessor)	The best free treatment of scheduling policy, and the source of the MLFQ rule formulation used here.
MIT 6.1810 — scheduling lecture and lab	xv6's scheduler in code, and the lab that replaces it.

Resource	What it covers for this module
Jones — What really happened on Mars (free)	The Pathfinder priority inversion, told by someone close to it. Short and worth reading in full.
Linux kernel documentation — CFS and EEVDF	How a production scheduler is actually structured.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Simulate FIFO, SJF, STCF, and round robin on the same set of jobs. Compute average turnaround and average response time for each and tabulate them.
- 2 Demonstrate the convoy effect: construct a job set where FIFO's average turnaround is many times SJF's.
- 3 Implement MLFQ in xv6, replacing the round-robin scheduler. Instrument it to record each process's queue level over time.
- 4 Remove rule 6 from your MLFQ and construct a workload that starves a long job. Then restore the rule and show the starvation disappears.
- 5 Write a process that games the scheduler by yielding just before its quantum expires. Demonstrate it monopolising the CPU, then implement cumulative accounting and show it no longer can.
- 6 Construct priority inversion in xv6 with three processes and a lock. Measure the high-priority process's delay, then implement priority inheritance and measure again.
- 7 Measure the cost of migrating a process between cores: pin a memory-intensive process to one core, then to another, and compare throughput immediately after the move.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Name five scheduling metrics and give a pair that directly conflict, with the reason.
- 2 Why is SJF optimal, and why is it unusable?
- 3 State the six MLFQ rules and say which two infer job type.
- 4 What goes wrong without the periodic priority boost?
- 5 How can a process game MLFQ, and what accounting change prevents it?
- 6 Describe priority inversion and explain how inheritance bounds it.
- 7 Why do multicore schedulers use per-CPU run queues, and what new trade does that create?